

Adding the **ICP-20111** ultra-low noise barometric pressure sensor opens up a whole new **3D spatial dimension** for your PDR tracking project.

Because the ICP-20111 has exceptional relative accuracy ($\pm 1 \text{ Pa}$), which corresponds to a vertical height resolution of roughly $\approx 8.5 \text{ cm}$, you can move from 2D plane mapping (X, Y) to true **3D Indoor Tracking (X, Y, Z)**.

1. What You Can Find Inside a Room Using ICP-20111

By integrating the barometer with your existing accelerometer + magnetometer setup, here are the exact parameters you can extract indoors:

A. Floor Level & Stair Climbing Detection (Z-axis Tracking)

- Floor Number Identification:** Easily detect when a user moves between floors in a building (e.g., Ground Floor $\rightarrow 1^{\text{st}}$ Floor $\rightarrow 2^{\text{nd}}$ Floor). Standard indoor floor heights are $3.0\text{--}3.5 \text{ meters}$ ($\approx 35\text{--}40 \text{ Pa}$ pressure drop), which the ICP-20111 detects effortlessly.

- Stair Step Counting:** Because of its 8.5 cm resolution, individual stair steps ($\approx 15\text{--}20 \text{ cm}$ height per riser) produce a distinct, step-like staircase pressure signal.

- Elevator / Lift Motion Detection:** Detects fast, smooth, continuous pressure changes when riding an elevator, allowing you to distinguish between walking up stairs versus riding a lift.

B. Micro-Level Vertical Actions Inside the Same Room

Even without leaving a single room, you can detect:

- Standing Up vs. Sitting Down:** Moving the sensor from table height (0.75 m) or pocket height (1.0 m) up to head height (1.7 m) produces a readable $\approx 8\text{--}11 \text{ Pa}$ pressure differential.

- Picking Up / Dropping Sensor:** Sudden height shifts of $30\text{--}50 \text{ cm}$ can be isolated from horizontal arm swings.

C. Environmental & Indoor Micro-Climate Data

- Ambient Room Pressure (P):** Measured directly in Pascals (Pa) or hectopascals (hPa).

- Indoor Ambient Temperature (T):** The ICP-20111 includes an internal calibrated temperature sensor used for thermal compensation, giving you exact room temperature readings.

- HVAC / Air Handling Activity:** Detecting minor pressure pulses when room doors

open/close or when central air conditioning turns on.

2. Using the Same Previous Connections (I²C Bus)

The ICP-20111 operates over standard **I²C**, meaning you **do not need extra micro-controller pins**. You connect it to the exact same I²C bus pins as your LSM303 sensor:

Bus Wiring Setup

- * **SDA Pin:** Connected to the same MicroPython I²C SDA pin (SDA_PIN = 0).
- * **SCL Pin:** Connected to the same MicroPython I²C SCL pin (SCL_PIN = 1).
- * **VCC / GND:** 3.3V power and Ground.
- * **I²C Address:** The ICP-20111 defaults to address **0x63** (or 0x64 depending on AD0 pin wiring). It sits alongside your LSM303 (0x19 ACC, 0x1E MAG) without any address conflicts!

3. Mathematical Formula for Room Height / Altitude

To convert the raw pressure output (P, in Pa) into relative indoor altitude (Δh , in meters) relative to your starting position in the room:

- * P_0 = Baseline atmospheric pressure recorded when you start the program in the room (Pa).
- * P = Current real-time filtered pressure reading (Pa).
- * Δh = Relative altitude offset in meters (Z-axis).

4. How This Enhances Your AI/ML Decision Tree

In your Spyder IDE model, you can now add **Vertical Velocity** ($\frac{\Delta h}{\Delta t}$) and **Pressure Variance** as a 5th feature to your dataset:

| Feature Input | What It Distinguishes |

|---|---|

| **std & p2p** | Distinguishes **Idle** vs **Walking** vs **Violent Cable Shaking**. |

| **pressure_delta (Δh)** | Distinguishes **Walking Flat in Room** vs **Climbing Stairs / Elevator**. |

5. MicroPython Code Snippet for ICP-20111

Here is how you initialize and read the ICP-20111 pressure sensor over the same I²C bus:

```
```python
from machine import Pin, I2C
import utime, math
```

```
ICP20111_ADDR = 0x63 # Default I2C Address for ICP-20111
```

```
Sensor Registers
```

```
REG_CMD_LOW = 0x0B
```

```
REG_CMD_HIGH = 0x00
```

```
def init_icp20111(i2c):
```

```
 # Send activation / measurement command to ICP-20111 (Ultra-low noise mode)
```

```
 try:
```

```
 i2c.writeto_mem(ICP20111_ADDR, 0x00, bytes([0x30, 0x0D]))
```

```
 utime.sleep_ms(10)
```

```
 print("ICP-20111 Barometer Initialized successfully!")
```

```
 except Exception as e:
```

```
 print("ICP-20111 Init Error:", e)
```

```
def read_pressure_height(i2c, baseline_p):
```

```
 try:
```

```
 # Read 3-byte raw pressure data
```

```
 data = i2c.readfrom_mem(ICP20111_ADDR, 0x06, 3)
```

```
 raw_p = (data[0] << 16) | (data[1] << 8) | data[2]
```

```
 # Scale to Pascals (ICP-20111 sensitivity factor)
```

```
 p_pascal = (raw_p / 7864320.0) * 40000.0 + 70000.0
```

```
 # Calculate Relative Height (Z-axis in meters)
```

```
 if baseline_p > 0:
```

```
 height = 44330.0 * (1.0 - math.pow((p_pascal / baseline_p), 0.1903))
```

```
 else:
```

```
 height = 0.0
```

```
 return p_pascal, height
```

```
 except:
```

```
return None, 0.0
```

```
...
```

### ### What To Update in Your Paper Title & Latex

If you include the ICP-20111 in your research paper, you can update your hardware methodology section to state:

```
> """The system uses a 3-axis accelerometer and magnetometer (LSM303) fused with
an ICP-20111 barometric pressure sensor to achieve full **3D Indoor Tracking (X, Y,
Z)** , detecting floor transitions ($\approx 3\text{ m}$) and stair elevation with an
accuracy resolution of 8.5 cm ."""
```

```
>
```

Here is the complete, end-to-end MicroPython solution for **3D Indoor Pedestrian Dead Reckoning (3D-PDR)**.

This program fuses your **3-Axis Accelerometer**, **3-Axis Magnetometer** (LSM303), and **ICP-20111 Barometer** over the same I<sup>2</sup>C bus. It includes **automatic runtime calibration** for all sensors, AI Decision Tree step-gating, dynamic direction tracking, 2D position (X, Y), and vertical height/floor tracking (Z).

### ## 1. What Exactly You Need to Do in the Setup

#### ### A. Hardware Wiring (Shared I<sup>2</sup>C Bus)

Connect both sensors to the exact same I<sup>2</sup>C pins on your microcontroller (e.g., Raspberry Pi Pico / ESP32):

Microcontroller Pin	LSM303 Sensor Pin	ICP-20111 Barometer Pin
---------------------	-------------------	-------------------------

---	---	---
-----	-----	-----

<b>3.3V Power</b>	VCC	VCC
-------------------	-----	-----

<b>GND Ground</b>	GND	GND
-------------------	-----	-----

<b>SDA (Pin 0)</b>	SDA	SDA
--------------------	-----	-----

<b>SCL (Pin 1)</b>	SCL	SCL
--------------------	-----	-----

*\*(Because I<sup>2</sup>C uses unique device addresses, LSM303 0x19/0x1E and ICP-20111 0x63 run simultaneously on the same two wires without conflicts).\**

#### ### B. Physical Calibration Procedure (Run when prompted on screen)

When you start the MicroPython program, the system will run a **10-second**

calibration routine\*\*:

1. **Accelerometer & Barometer Calibration (First 3 seconds):** Keep the sensor **completely flat and still** on a table. The script measures gravity bias and locks in the baseline room atmospheric pressure ( $P_0 = \text{Ground Floor Height}$ ).

2. **Magnetometer Calibration (Next 7 seconds):** Rotate the sensor in a slow **"Figure-8"** motion in the air. This calculates hard-iron distortion offsets so your compass heading ( $\text{N, S, E, W}$ ) and turn angles remain accurate indoors.

## ## 2. Complete MicroPython 3D-PDR Script

Save and run this script as **main.py** on your board:

```
```python
#
=====
=====
# COMPLETE 3D INDOOR TRACKING & MAPPING PROGRAM (PDR + AI +
BAROMETER)
# Sensors: LSM303 (Accel + Mag) & ICP-20111 (Barometer)
# Features: 2D Position (X,Y), Compass Heading, Floor/Height (Z), AI Gating
#
=====
=====
from machine import Pin, I2C
import utime, math

# --- I2C CONFIGURATION ---
I2C_ID = 0
SDA_PIN = 0
SCL_PIN = 1
I2C_FREQ = 100000

# Sensor I2C Addresses
ACC_ADDRESS = 0x19
MAG_ADDRESS = 0x1E
BARO_ADDRESS = 0x63
```

LSM303 Registers

CTRL_REG1_A = 0x20

CTRL_REG4_A = 0x23

OUT_X_L_A = 0x28

CRA_REG_M = 0x00

CRB_REG_M = 0x01

MR_REG_M = 0x02

OUT_X_H_M = 0x03

Tuning Parameters

STRIDE_LENGTH_M = 0.50 # Average step length in meters

STEP_COOLDOWN_MS = 380 # Min time between human steps

ALPHA = 0.30 # EMA Low-pass filter smoothing coefficient

FLOOR_HEIGHT_M = 3.0 # Height per building floor in meters

Calibration Offset Variables

x_bias, y_bias, z_bias = 0.0, 0.0, 0.0

mag_x_bias, mag_y_bias = 0.0, 0.0

baseline_pressure = 0.0 # Ground pressure baseline (Pa)

System Navigation State

total_steps = 0

total_distance = 0.0

pos_x, pos_y, pos_z = 0.0, 0.0, 0.0

current_floor = 0

last_step_time = 0

last_heading = 0.0

prev_activity = "UNKNOWN"

i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))

I2C HELPER FUNCTIONS

```
def write_reg(addr, reg, val):
```

```
    try:
```

```
        i2c.writeto_mem(addr, reg, bytes([val]))
```

```
    except: pass
```

```
def read_regs(addr, reg, length):
```

```
    try:
```

```
        return i2c.readfrom_mem(addr, reg, length)
```

```
    except: return None
```

```
def signed_12bit(val):
```

```
    val >>= 4
```

```
    if val >= 2048: val -= 4096
```

```
    return val
```

```
def signed_16bit(val):
```

```
    if val >= 32768: val -= 65536
```

```
    return val
```

SENSOR INITIALIZATION & READERS

```
def init_sensors():
```

```
    # Accelerometer (100Hz ODR, High Res)
```

```
    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57)
```

```
    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08)
```

```
    # Magnetometer (30Hz ODR, Continuous)
```

```
    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14)
```

```
    write_reg(MAG_ADDRESS, CRB_REG_M, 0x20)
```

```
    write_reg(MAG_ADDRESS, MR_REG_M, 0x00)
```

ICP-20111 Barometer (Activation Command)

try:

i2c.writeto_mem(BARO_ADDRESS, 0x00, bytes([0x30, 0x0D]))

utime.sleep_ms(10)

except:

print("[!] Barometer warning: Check ICP-20111 wiring.")

def read_accel():

data = read_regs(ACC_ADDRESS, OUT_X_L_A | 0x80, 6)

if data is None: return None

x = signed_12bit((data[1] << 8) | data[0])

y = signed_12bit((data[3] << 8) | data[2])

z = signed_12bit((data[5] << 8) | data[4])

return x, y, z

def read_mag():

data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)

if data is None: return None

mx = signed_16bit((data[0] << 8) | data[1])

mz = signed_16bit((data[2] << 8) | data[3])

my = signed_16bit((data[4] << 8) | data[5])

return mx, my, mz

def read_baro_pressure():

try:

data = i2c.readfrom_mem(BARO_ADDRESS, 0x06, 3)

raw_p = (data[0] << 16) | (data[1] << 8) | data[2]

ICP-20111 Pressure conversion to Pascals

p_pascal = (raw_p / 7864320.0) * 40000.0 + 70000.0

return p_pascal

except:

return None

```

# -----
# AUTOMATIC SENSOR CALIBRATION ROUTINE
# -----

def calibrate_system():
    global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias, baseline_pressure

    print("\n=====")
    print("STEP 1/2: ACCEL & BARO CALIBRATION (Keep sensor STILL)")
    print("=====")
    sx, sy, sz, p_sum, count = 0, 0, 0, 0, 0

    for _ in range(100):
        acc = read_accel()
        p = read_baro_pressure()
        if acc and p:
            sx += acc[0]
            sy += acc[1]
            sz += acc[2]
            p_sum += p
            count += 1
        utime.sleep_ms(30)

    x_bias = sx / count
    y_bias = sy / count
    z_bias = (sz / count) - 1024.0 # Remove 1G gravity
    baseline_pressure = p_sum / count

    print(f"-> Accel Biases: X={x_bias:.1f}, Y={y_bias:.1f}, Z={z_bias:.1f}")
    print(f"-> Baseline Room Pressure: {baseline_pressure:.2f} Pa")

```

```

print("\n=====")
print("STEP 2/2: MAGNETOMETER CALIBRATION (Rotate in Figure-8)")
print("=====")
min_x, max_x = 32767, -32768
min_y, max_y = 32767, -32768

start_time = utime.ticks_ms()
while utime.ticks_diff(utime.ticks_ms(), start_time) < 7000: # 7 sec sweep
    m = read_mag()
    if m:
        if m[0] < min_x: min_x = m[0]
        if m[0] > max_x: max_x = m[0]
        if m[1] < min_y: min_y = m[1]
        if m[1] > max_y: max_y = m[1]
    utime.sleep_ms(20)

mag_x_bias = (max_x + min_x) / 2.0
mag_y_bias = (max_y + min_y) / 2.0
print(f"-> Mag Biases: X={mag_x_bias:.1f}, Y={mag_y_bias:.1f}")

print("=====\n")

# -----
# HEADING & ALTITUDE MATH
# -----

def get_heading():
    m = read_mag()
    if m is None: return 0.0
    mx_c = m[0] - mag_x_bias
    my_c = m[1] - mag_y_bias
    heading = math.degrees(math.atan2(my_c, mx_c))
    if heading < 0: heading += 360.0
    return heading

```

```
def get_cardinal(h):
    if h >= 337.5 or h < 22.5: return "NORTH"
    elif 22.5 <= h < 67.5: return "NORTH-EAST"
    elif 67.5 <= h < 112.5: return "EAST"
    elif 112.5 <= h < 157.5: return "SOUTH-EAST"
    elif 157.5 <= h < 202.5: return "SOUTH"
    elif 202.5 <= h < 247.5: return "SOUTH-WEST"
    elif 247.5 <= h < 292.5: return "WEST"
    else: return "NORTH-WEST"
```

```
def get_relative_altitude(p_current):
    if p_current is None or baseline_pressure == 0: return 0.0
    # Hypsometric formula for indoor height change (Meters)
    h = 44330.0 * (1.0 - math.pow((p_current / baseline_pressure), 0.1903))
    return h
```

AI Decision Tree Classifier Gating

```
def ai_classify_motion(accel, avg, std, p2p):
    if std <= 1.8450:
        return 0 # IDLE / SITTING
    else:
        if p2p > 82.5000:
            return 2 # SHAKING / CABLE NOISE
        else:
            if accel > (avg * 1.15):
                return 1 # VALID WALKING
            else:
                return 0 # RECOVERY
```

MAIN EXECUTION LOOP

```

def main():
    global total_steps, total_distance, last_step_time, pos_x, pos_y, pos_z
    global current_floor, last_heading, prev_activity

    init_sensors()
    calibrate_system()

    smooth_accel = 0.0
    accel_buffer = []

    print(">>> 3D-PDR SYSTEM ONLINE: START WALKING <<<\n")

    while True:
        now = utime.ticks_ms()
        acc = read_accel()

        if acc:
            ax = acc[0] - x_bias
            ay = acc[1] - y_bias
            az = acc[2] - z_bias

            # Magnitude & Low-Pass Filter
            raw_mag = math.sqrt(ax**2 + ay**2 + az**2)
            smooth_accel = (ALPHA * raw_mag) + ((1.0 - ALPHA) * smooth_accel)

            # Maintain 15-sample rolling buffer
            accel_buffer.append(smooth_accel)
            if len(accel_buffer) > 15: accel_buffer.pop(0)

            if len(accel_buffer) == 15:
                # 1. Extract Statistical Features
                avg_val = sum(accel_buffer) / 15.0
                variance = sum((x - avg_val)**2 for x in accel_buffer) / 15.0

```

```
std_dev = math.sqrt(variance)
p2p_delta = max(accel_buffer) - min(accel_buffer)
```

```
# 2. Run AI Inference
```

```
motion_state = ai_classify_motion(smooth_accel, avg_val, std_dev,
p2p_delta)
```

```
# Activity Label Update
```

```
if motion_state == 1: activity = "WALKING"
elif motion_state == 2: activity = "SHAKING/NOISE"
else: activity = "IDLE/SITTING"
```

```
if activity != prev_activity:
    print(f"\n[ACTIVITY STATE CHANGE]: {activity}")
    prev_activity = activity
```

```
# 3. Read Altitude & Floor Level from ICP-20111
```

```
current_p = read_baro_pressure()
pos_z = get_relative_altitude(current_p)
current_floor = int(round(pos_z / FLOOR_HEIGHT_M))
```

```
# 4. Step Detection & Tracking Update (AI-Gated)
```

```
if motion_state == 1: # Only execute if AI verifies WALKING
```

```
    time_diff = utime.ticks_diff(now, last_step_time)
```

```
    if time_diff > STEP_COOLDOWN_MS:
```

```
        total_steps += 1
```

```
        total_distance += STRIDE_LENGTH_M
```

```
        last_step_time = now
```

```
    heading = get_heading()
```

```
    cardinal = get_cardinal(heading)
```

```
# Turn Detection
```

```

heading_diff = abs(heading - last_heading)
if heading_diff > 180: heading_diff = 360 - heading_diff
if last_heading != 0 and heading_diff > 25:
    turn = "RIGHT" if (heading - last_heading) % 360 < 180 else "LEFT"
    print(f"  [! TURN DETECTED]: {turn} by {heading_diff:.1f}°")
last_heading = heading

```

```

# 2D Position Trigonometry (X, Y)
rad = math.radians(heading)
pos_x += STRIDE_LENGTH_M * math.sin(rad)
pos_y += STRIDE_LENGTH_M * math.cos(rad)

```

```

# Output Complete Live Tracking Status
print(f"STEP #{total_steps} | Dir: {cardinal} ({heading:.1f}°) | "
      f"Dist: {total_distance:.1f}m | "
      f"Pos(X,Y): ({pos_x:.2f}m, {pos_y:.2f}m) | "
      f"Height(Z): {pos_z:.2f}m (Floor {current_floor})")

```

```

utime.sleep_ms(20)

```

```

if __name__ == "__main__":
    main()

```

```

...

```

3. What the Real-Time Terminal Output Looks Like

When you walk, turn, or change floor levels, the system prints live 3D tracking coordinates:

```

```text

```

```

=====

```

```

STEP 1/2: ACCEL & BARO CALIBRATION (Keep sensor STILL)

```

```

=====

```

```

-> Accel Biases: X=12.4, Y=-8.1, Z=15.2

```

```

-> Baseline Room Pressure: 101325.40 Pa

```

=====

STEP 2/2: MAGNETOMETER CALIBRATION (Rotate in Figure-8)

=====

-> Mag Biases: X=142.5, Y=-98.0

=====

>>> 3D-PDR SYSTEM ONLINE: START WALKING <<<

[ACTIVITY STATE CHANGE]: WALKING

STEP #1 | Dir: NORTH (12.4°) | Dist: 0.5m | Pos(X,Y): (0.11m, 0.49m) | Height(Z):  
+0.05m (Floor 0)

STEP #2 | Dir: NORTH (13.1°) | Dist: 1.0m | Pos(X,Y): (0.22m, 0.98m) | Height(Z):  
+0.08m (Floor 0)

[! TURN DETECTED]: RIGHT by 88.5°

STEP #3 | Dir: EAST (101.6°) | Dist: 1.5m | Pos(X,Y): (0.71m, 0.88m) | Height(Z):  
+0.12m (Floor 0)

[ACTIVITY STATE CHANGE]: SHAKING/NOISE <-- (Cable shaking or tap ignored!)

[ACTIVITY STATE CHANGE]: WALKING (CLIMBING STAIRS)

STEP #4 | Dir: EAST (100.2°) | Dist: 2.0m | Pos(X,Y): (1.20m, 0.79m) | Height(Z):  
+1.52m (Floor 1)

STEP #5 | Dir: EAST (101.0°) | Dist: 2.5m | Pos(X,Y): (1.69m, 0.70m) | Height(Z):  
+3.04m (Floor 1)

...

Here is the complete **\*\*Standalone (Non-AI) MicroPython Program\*\*** for 3D Indoor  
Pedestrian Dead Reckoning (PDR).

This version uses heuristic thresholding (fixed mathematical thresholds) without any machine learning classifiers to process the **Accelerometer**, **Magnetometer** (LSM303), and **ICP-20111 Barometer** on the same I<sup>2</sup>C bus.

### Complete Standalone MicroPython Script

```
```python
```

```
#
```

```
=====
```

```
=====
```

```
# STANDALONE (NON-AI) 3D INDOOR TRACKING & MAPPING PROGRAM
```

```
# Hardware: LSM303 (Accel + Mag) & ICP-20111 (Barometer)
```

```
# Features: Peak Threshold Step Detection, Compass Heading, Height/Floor Level
```

```
#
```

```
=====
```

```
=====
```

```
from machine import Pin, I2C
```

```
import utime, math
```

```
# --- I2C BUS CONFIGURATION ---
```

```
I2C_ID = 0
```

```
SDA_PIN = 0
```

```
SCL_PIN = 1
```

```
I2C_FREQ = 100000
```

```
ACC_ADDRESS = 0x19
```

```
MAG_ADDRESS = 0x1E
```

```
BARO_ADDRESS = 0x63
```

```
# Registers
```

```
CTRL_REG1_A = 0x20
```

```
CTRL_REG4_A = 0x23
```

```
OUT_X_L_A = 0x28
```

```
CRA_REG_M = 0x00
```

```
CRB_REG_M = 0x01
```

```
MR_REG_M = 0x02
OUT_X_H_M = 0x03
```

```
# --- HEURISTIC TUNING PARAMETERS ---
```

```
STRIDE_LENGTH_M = 0.50    # Distance per step in meters
STEP_COOLDOWN_MS = 380    # Minimum time delay between steps
ACCEL_THRESHOLD = 150.0   # Heuristic step peak threshold
ALPHA = 0.30              # EMA Smoothing Factor
FLOOR_HEIGHT_M = 3.0      # Building floor height in meters
```

```
# Sensor Offsets & Baseline Variables
```

```
x_bias, y_bias, z_bias = 0.0, 0.0, 0.0
mag_x_bias, mag_y_bias = 0.0, 0.0
baseline_pressure = 0.0
```

```
# Tracking State
```

```
total_steps = 0
total_distance = 0.0
pos_x, pos_y, pos_z = 0.0, 0.0, 0.0
current_floor = 0
last_step_time = 0
last_heading = 0.0
```

```
i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))
```

```
# --- I2C LOW-LEVEL DRIVERS ---
```

```
def write_reg(addr, reg, val):
    try:
        i2c.writeto_mem(addr, reg, bytes([val]))
    except: pass
```

```
def read_regs(addr, reg, length):
    try:
```

```

        return i2c.readfrom_mem(addr, reg, length)
    except: return None

def signed_12bit(val):
    val >>= 4
    if val >= 2048: val -= 4096
    return val

def signed_16bit(val):
    if val >= 32768: val -= 65536
    return val

# --- SENSOR INIT & READERS ---
def init_sensors():
    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57) # Accel 100Hz ODR
    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08) # High Resolution
    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14) # Mag 30Hz ODR
    write_reg(MAG_ADDRESS, CRB_REG_M, 0x20) # Gain
    write_reg(MAG_ADDRESS, MR_REG_M, 0x00) # Continuous
    try:
        i2c.writeto_mem(BARO_ADDRESS, 0x00, bytes([0x30, 0x0D])) # Start ICP-
20111
        utime.sleep_ms(10)
    except: pass

def read_accel():
    data = read_regs(ACC_ADDRESS, OUT_X_L_A | 0x80, 6)
    if data is None: return None
    return signed_12bit((data[1]<<8)|data[0]), signed_12bit((data[3]<<8)|data[2]),
signed_12bit((data[5]<<8)|data[4])

def read_mag():
    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)

```

```
    if data is None: return None
    return signed_16bit((data[0]<<8)|data[1]), signed_16bit((data[2]<<8)|data[3]),
signed_16bit((data[4]<<8)|data[5])
```

```
def read_baro_pressure():
    try:
        data = i2c.readfrom_mem(BARO_ADDRESS, 0x06, 3)
        raw_p = (data[0] << 16) | (data[1] << 8) | data[2]
        return (raw_p / 7864320.0) * 40000.0 + 70000.0
    except: return None
```

```
# --- SENSOR CALIBRATION ---
```

```
def calibrate_sensors():
    global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias, baseline_pressure
    print("Calibrating... Keep sensor STILL on desk.")
    sx, sy, sz, p_sum, count = 0, 0, 0, 0, 0
    for _ in range(100):
        a = read_accel()
        p = read_baro_pressure()
        if a and p:
            sx += a[0]; sy += a[1]; sz += a[2]
            p_sum += p; count += 1
        utime.sleep_ms(20)
    x_bias, y_bias = sx / count, sy / count
    z_bias = (sz / count) - 1024.0 # Remove gravity vector
    baseline_pressure = p_sum / count

    print("Rotate sensor in a Figure-8 shape in air...")
    min_x, max_x, min_y, max_y = 32767, -32768, 32767, -32768
    start = utime.ticks_ms()
    while utime.ticks_diff(utime.ticks_ms(), start) < 6000:
        m = read_mag()
        if m:
```

```
    if m[0] < min_x: min_x = m[0]
    if m[0] > max_x: max_x = m[0]
    if m[1] < min_y: min_y = m[1]
    if m[1] > max_y: max_y = m[1]
    utime.sleep_ms(20)
    mag_x_bias = (max_x + min_x) / 2.0
    mag_y_bias = (max_y + min_y) / 2.0
    print("Calibration Complete!\n")
```

```
# --- NAVIGATION MATH ---
```

```
def get_heading():
    m = read_mag()
    if m is None: return 0.0
    mx_c = m[0] - mag_x_bias
    my_c = m[1] - mag_y_bias
    heading = math.degrees(math.atan2(my_c, mx_c))
    return heading + 360.0 if heading < 0 else heading
```

```
def get_cardinal(h):
    if h >= 337.5 or h < 22.5: return "N"
    elif 22.5 <= h < 67.5: return "NE"
    elif 67.5 <= h < 112.5: return "E"
    elif 112.5 <= h < 157.5: return "SE"
    elif 157.5 <= h < 202.5: return "S"
    elif 202.5 <= h < 247.5: return "SW"
    elif 247.5 <= h < 292.5: return "W"
    else: return "NW"
```

```
def get_relative_altitude(p_curr):
    if p_curr is None or baseline_pressure == 0: return 0.0
    return 44330.0 * (1.0 - math.pow((p_curr / baseline_pressure), 0.1903))
```

```
# --- MAIN EXECUTION LOOP ---
```

```

def main():
    global total_steps, total_distance, last_step_time, pos_x, pos_y, pos_z,
    current_floor, last_heading
    init_sensors()
    calibrate_sensors()

    smooth_accel = 0.0
    accel_buffer = []

    print("--- Standalone PDR Running ---")

    while True:
        now = utime.ticks_ms()
        acc = read_accel()

        if acc:
            ax, ay, az = acc[0] - x_bias, acc[1] - y_bias, acc[2] - z_bias

            # 1. Acceleration Vector Magnitude Calculation
            raw_mag = math.sqrt(ax**2 + ay**2 + az**2)

            # 2. Low-Pass Exponential Moving Average (EMA) Filtering
            smooth_accel = (ALPHA * raw_mag) + ((1.0 - ALPHA) * smooth_accel)

            # Maintain 15-sample rolling buffer
            accel_buffer.append(smooth_accel)
            if len(accel_buffer) > 15: accel_buffer.pop(0)

            if len(accel_buffer) == 15:
                avg_buffer = sum(accel_buffer) / 15.0

            # 3. Barometric Altitude Update
            p_curr = read_baro_pressure()

```

```
pos_z = get_relative_altitude(p_curr)
current_floor = int(round(pos_z / FLOOR_HEIGHT_M))
```

```
# 4. Standalone Peak Thresholding Step Detection
```

```
# Condition A: Force peak higher than static threshold
```

```
# Condition B: Peak is 40% higher than the local average
```

```
# Condition C: Minimum time delay between steps met
```

```
if (smooth_accel > ACCEL_THRESHOLD) and (smooth_accel >
(avg_buffer * 1.40)):
```

```
    time_diff = utime.ticks_diff(now, last_step_time)
```

```
    if time_diff > STEP_COOLDOWN_MS:
```

```
        total_steps += 1
```

```
        total_distance += STRIDE_LENGTH_M
```

```
        last_step_time = now
```

```
# Heading & Compass Direction
```

```
heading = get_heading()
```

```
cardinal = get_cardinal(heading)
```

```
# Turn Detection
```

```
heading_diff = abs(heading - last_heading)
```

```
if heading_diff > 180: heading_diff = 360 - heading_diff
```

```
if last_heading != 0 and heading_diff > 25:
```

```
    turn = "RIGHT" if (heading - last_heading) % 360 < 180 else "LEFT"
```

```
    print(f"  [!] TURN DETECTED: {turn} ({heading_diff:.1f}°)")
```

```
last_heading = heading
```

```
# 2D Cartesian Coordinate Update (X, Y)
```

```
rad = math.radians(heading)
```

```
pos_x += STRIDE_LENGTH_M * math.sin(rad)
```

```
pos_y += STRIDE_LENGTH_M * math.cos(rad)
```

```
print(f"STEP #{total_steps} | Dir: {cardinal} ({heading:.1f}°) | "
```

```
f"Dist: {total_distance:.1f}m | "
f"Pos(X,Y): ({pos_x:.2f}m, {pos_y:.2f}m) | "
f"Height(Z): {pos_z:+.2f}m (Floor {current_floor})")
```

```
utime.sleep_ms(20)
```

```
if __name__ == "__main__":
    main()
```

```
...
```

Technical Explanation of the Standalone Program

The standalone code relies strictly on **direct sensor signals** and **fixed mathematical logic** without any machine learning classification.

1. Signal Conditioning & Filtering

* **Raw Magnitude Calculation:** Computes the 3D acceleration vector length $a_{\text{mag}} = \sqrt{a_x^2 + a_y^2 + a_z^2}$ to make step detection independent of sensor orientation.

* **EMA Low-Pass Filter:** Uses an Exponential Moving Average ($S_t = \alpha \cdot X_t + (1 - \alpha) \cdot S_{t-1}$) with $\alpha = 0.30$ to smooth out high-frequency electrical and mechanical noise.

2. Standalone Peak Thresholding (Step Counting Logic)

A step is counted when all three heuristic rules are met simultaneously:

1. **Magnitude Condition:** $\text{smooth_accel} > \text{ACCEL_THRESHOLD}$ (Checks if force exceeds 150.0 LSB).

2. **Relative Peak Condition:** $\text{smooth_accel} > \text{avg_buffer} \cdot 1.40$ (Ensures the current force spike is at least 40% higher than the average force over the last 15 samples).

3. **Refractory Cooldown:** $\text{time_diff} > 380 \text{ ms}$ (Prevents double-counting the recoil from a single step).

3. Heading & Direction Tracking

* **Hard-Iron Magnetometer Calibration:** Calibrates baseline magnetic offsets ($M_{x,\text{bias}}$, $M_{y,\text{bias}}$) using a 6-second Figure-8 rotation sweep.

* **Azimuth Math:** Calculates absolute heading in degrees using $\theta =$

$\text{atan2}(M_y - M_{y,\text{bias}}, M_x - M_{x,\text{bias}})$.

* **2D Dead Reckoning (X, Y):** Updates Cartesian coordinates using trigonometry upon every validated step:

4. Barometric Altitude Tracking (Z-axis)

* **Baseline Atmospheric Lock:** Locks in baseline atmospheric ground pressure (P_0) during static initialization.

* **Hypsometric Altitude Formula:** Converts real-time pressure (P) into vertical elevation in meters:

* **Floor Level Mapping:** Calculates the building floor index by dividing relative height by the standard floor height (3.0 m): $\text{Floor} = \lfloor \frac{\Delta h}{3.0} \rfloor$.

Key Limitation of Standalone Mode

Because the step detection rules rely entirely on static magnitude thresholds ($\text{smooth_accel} > 150$), **non-walking motion—such as tapping on a desk, shaking the USB cable, or quick hand movements—will trigger false steps**, leading to accumulated distance errors over time.

Here is the complete **AI/ML-Enhanced 3D Indoor Tracking & Mapping Solution**.

By adding an AI Decision Tree classifier trained in **Spyder IDE** via scikit-learn, we solve the primary weakness of the standalone code. Instead of relying on a simple threshold ($\text{smooth_accel} > 150$) that gets tricked by hand fidgeting, table taps, or cable shaking, the AI classifier evaluates **Rolling Mean**, **Rolling Standard Deviation**, and **Peak-to-Peak Delta** to ensure steps are **only counted when real walking motion occurs**.

Part 1: End-to-End Workflow (What You Need to Do)

Step 1: Collect Data for AI Training (MicroPython)

1. Flash the **Data Collection Script** (provided in Part 2 below) onto your microcontroller.
2. Open your MicroPython REPL Terminal (Thonny IDE / Serial Monitor).

3. Record 3 separate CSV datasets by copying the REPL output:

* **idle.csv**: Keep the sensor flat/still or held loosely while sitting for 10–15 seconds. (Label: 0)

* **walking.csv**: Walk normally with the natural arm-swing motion for 15–20 seconds. (Label: 1)

* **shaking.csv**: Shake the cable, tap the table, or move your hands wildly for 10–15 seconds. (Label: 2)

Step 2: Train Model & Auto-Generate Code in Spyder IDE (PC)

1. Place idle.csv, walking.csv, and shaking.csv in your Spyder project folder.
2. Run the **Spyder Training Script** (provided in Part 3 below).
3. The script will train a Decision Tree (max_depth=3) and print an **auto-generated MicroPython function** (def ai_classify_motion(...)) directly in the Spyder console!

Step 3: Run the AI-Enhanced 3D-PDR Script on MCU

1. Copy the auto-generated ai_classify_motion function from Spyder and paste it into the **Main AI-Enhanced MicroPython Program** (Part 4 below).
2. Save and run the script as main.py on your microcontroller.
3. Perform the **10-second calibration** when prompted:

* **First 3 seconds**: Keep sensor flat and still (calibrates accelerometer gravity & baseline pressure P_0).

* **Next 7 seconds**: Rotate sensor in a slow Figure-8 in the air (calibrates magnetometer hard-iron offsets).

4. Start walking! The system will give real-time 3D coordinates (X, Y, Z), compass heading, floor level, and block false step triggers.

Part 2: Data Collection Script (MicroPython)

Flash this script to record sensor feature extractions over REPL:

```
```python
```

```
#
```

```
=====
```

```
=====
```

```
DATA COLLECTION SCRIPT (Run on MCU to gather training CSVs)
```

```
#
```

```
=====
```

```
=====
```

```

from machine import Pin, I2C
import utime, math

i2c = I2C(0, freqs=100000, sda=Pin(0), scl=Pin(1))

def write_reg(addr, reg, val):
 try: i2c.writeto_mem(addr, reg, bytes([val]))
 except: pass

def read_regs(addr, reg, length):
 try: return i2c.readfrom_mem(addr, reg, length)
 except: return None

def signed_12bit(val):
 val >>= 4
 if val >= 2048: val -= 4096
 return val

write_reg(0x19, 0x20, 0x57) # Accel 100Hz ODR
write_reg(0x19, 0x23, 0x08) # High resolution

print("--- DATA COLLECTION MODE STARTED ---")
print("Format: smooth_accel,avg_val,std_dev,p2p_delta")

buffer = []
ALPHA = 0.30
smooth_accel = 0.0

while True:
 data = read_regs(0x19, 0x28 | 0x80, 6)
 if data:
 x = signed_12bit((data[1] << 8) | data[0])
 y = signed_12bit((data[3] << 8) | data[2])

```

```
z = signed_12bit((data[5] << 8) | data[4])
```

```
raw_mag = math.sqrt(x**2 + y**2 + z**2)
```

```
smooth_accel = (ALPHA * raw_mag) + ((1.0 - ALPHA) * smooth_accel)
```

```
buffer.append(smooth_accel)
```

```
if len(buffer) > 15: buffer.pop(0)
```

```
if len(buffer) == 15:
```

```
 avg_val = sum(buffer) / 15.0
```

```
 variance = sum((v - avg_val)**2 for v in buffer) / 15.0
```

```
 std_dev = math.sqrt(variance)
```

```
 p2p_delta = max(buffer) - min(buffer)
```

```
 # Print comma-separated features to copy into CSV
```

```
 print(f'{smooth_accel:.2f},{avg_val:.2f},{std_dev:.2f},{p2p_delta:.2f}')
```

```
 utime.sleep_ms(20)
```

```
...
```

```
Part 3: Spyder Training & MicroPython Code Generator Script (PC)
```

Run this in **Spyder IDE** to train the model and generate the embedded

MicroPython Decision Tree code:

```
```python
```

```
#
```

```
=====
```

```
=====
```

```
# SPYDER SCRIPT: TRAIN MODEL & AUTO-GENERATE EMBEDDED
```

```
MICROPYTHON CODE
```

```
#
```

```
=====
```

```
=====
```

```
import pandas as pd
```

```

import numpy as np
from sklearn.tree import DecisionTreeClassifier, _tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Load CSVs (Ensure files are in current working directory)
df_idle = pd.read_csv('idle.csv', names=['accel', 'avg', 'std', 'p2p'])
df_idle['label'] = 0

df_walk = pd.read_csv('walking.csv', names=['accel', 'avg', 'std', 'p2p'])
df_walk['label'] = 1

df_shake = pd.read_csv('shaking.csv', names=['accel', 'avg', 'std', 'p2p'])
df_shake['label'] = 2

df = pd.concat([df_idle, df_walk, df_shake], ignore_index=True)

X = df[['accel', 'avg', 'std', 'p2p']]
y = df['label']

# 2. Train Model
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)

print(f"Model Training Accuracy: {accuracy_score(y_test,
clf.predict(X_test))*100:.2f}%\n")

# 3. Auto-Generator Function
def export_micropython_code(tree, feature_names):
    tree_ = tree.tree_
    feature_name = [

```

```

        feature_names[i] if i != _tree.TREE_UNDEFINED else "undefined!"
    for i in tree_.feature
]

code_lines = []
code_lines.append("# --- AUTO-GENERATED MICROPYTHON DECISION TREE
ENGINE ---")
code_lines.append("def ai_classify_motion(accel, avg, std, p2p):")

def recurse(node, depth):
    indent = "    " * depth
    if tree_.feature[node] != _tree.TREE_UNDEFINED:
        name = feature_name[node]
        threshold = tree_.threshold[node]
        code_lines.append(f'{indent}if {name} <= {threshold:.4f}:')
        recurse(tree_.children_left[node], depth + 1)
        code_lines.append(f'{indent}else:')
        recurse(tree_.children_right[node], depth + 1)
    else:
        class_idx = np.argmax(tree_.value[node])
        code_lines.append(f'{indent}return {class_idx}')

recurse(0, 1)
return "\n".join(code_lines)

print("=====
=====")
print("  COPY EVERYTHING BELOW THIS LINE DIRECTLY INTO YOUR
MICROPYTHON PROGRAM  ")
print("=====
=====")
print(export_micropython_code(clf, ['accel', 'avg', 'std', 'p2p']))
print("=====
=====")

```

```
=====")
```

```
...
```

```
## Part 4: Complete AI-Enhanced 3D-PDR MicroPython Program
```

```
This script combines **LSM303 (Accel + Mag)**, **ICP-20111 Barometer**, automatic calibration, 3D tracking math, and the **AI Decision Tree Inference Engine**.
```

```
```python
```

```
#
```

```
=====
```

```
=====
```

```
COMPLETE AI-ENHANCED 3D INDOOR TRACKING & MAPPING PROGRAM
```

```
Sensors: LSM303 (Accel + Mag) & ICP-20111 (Barometer)
```

```
Features: AI Motion Gating, 2D Position (X,Y), Altitude & Floor Level (Z)
```

```
#
```

```
=====
```

```
=====
```

```
from machine import Pin, I2C
```

```
import utime, math
```

```
--- I2C CONFIGURATION ---
```

```
I2C_ID = 0
```

```
SDA_PIN = 0
```

```
SCL_PIN = 1
```

```
I2C_FREQ = 100000
```

```
ACC_ADDRESS = 0x19
```

```
MAG_ADDRESS = 0x1E
```

```
BARO_ADDRESS = 0x63
```

```
Registers
```

```
CTRL_REG1_A = 0x20
```

```
CTRL_REG4_A = 0x23
```

```
OUT_X_L_A = 0x28
```

```
CRA_REG_M = 0x00
```

```
CRB_REG_M = 0x01
```

```
MR_REG_M = 0x02
```

```
OUT_X_H_M = 0x03
```

```
Tuning Parameters
```

```
STRIDE_LENGTH_M = 0.50 # Average step length in meters
```

```
STEP_COOLDOWN_MS = 380 # Min time between steps
```

```
ALPHA = 0.30 # EMA Low-pass filter smoothing coefficient
```

```
FLOOR_HEIGHT_M = 3.0 # Building floor height in meters
```

```
Calibration Offsets
```

```
x_bias, y_bias, z_bias = 0.0, 0.0, 0.0
```

```
mag_x_bias, mag_y_bias = 0.0, 0.0
```

```
baseline_pressure = 0.0
```

```
Tracking State
```

```
total_steps = 0
```

```
total_distance = 0.0
```

```
pos_x, pos_y, pos_z = 0.0, 0.0, 0.0
```

```
current_floor = 0
```

```
last_step_time = 0
```

```
last_heading = 0.0
```

```
prev_activity = "UNKNOWN"
```

```
i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))
```

```
--- LOW LEVEL I2C DRIVERS ---
```

```
def write_reg(addr, reg, val):
```

```
 try: i2c.writeto_mem(addr, reg, bytes([val]))
```

```
 except: pass
```

```
def read_regs(addr, reg, length):
```

```
try: return i2c.readfrom_mem(addr, reg, length)
except: return None
```

```
def signed_12bit(val):
 val >>= 4
 if val >= 2048: val -= 4096
 return val
```

```
def signed_16bit(val):
 if val >= 32768: val -= 65536
 return val
```

```
--- SENSOR INIT & READERS ---
```

```
def init_sensors():
 write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57) # Accel 100Hz ODR
 write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08) # High Res
 write_reg(MAG_ADDRESS, CRA_REG_M, 0x14) # Mag 30Hz ODR
 write_reg(MAG_ADDRESS, CRB_REG_M, 0x20) # Gain
 write_reg(MAG_ADDRESS, MR_REG_M, 0x00) # Continuous
 try:
 i2c.writeto_mem(BARO_ADDRESS, 0x00, bytes([0x30, 0x0D])) # Start
 Barometer
 utime.sleep_ms(10)
 except: pass
```

```
def read_accel():
 data = read_regs(ACC_ADDRESS, OUT_X_L_A | 0x80, 6)
 if data is None: return None
 return signed_12bit((data[1]<<8)|data[0]), signed_12bit((data[3]<<8)|data[2]),
 signed_12bit((data[5]<<8)|data[4])
```

```
def read_mag():
 data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)
```

```

 if data is None: return None

 return signed_16bit((data[0]<<8)|data[1]), signed_16bit((data[2]<<8)|data[3]),
 signed_16bit((data[4]<<8)|data[5])

def read_baro_pressure():
 try:
 data = i2c.readfrom_mem(BARO_ADDRESS, 0x06, 3)
 raw_p = (data[0] << 16) | (data[1] << 8) | data[2]
 return (raw_p / 7864320.0) * 40000.0 + 70000.0
 except: return None

--- SENSOR CALIBRATION ROUTINE ---
def calibrate_system():
 global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias, baseline_pressure
 print("=====")
 print("STEP 1/2: ACCEL & BARO CALIBRATION (Keep sensor STILL)")
 print("=====")
 sx, sy, sz, p_sum, count = 0, 0, 0, 0, 0
 for _ in range(100):
 a = read_accel()
 p = read_baro_pressure()
 if a and p:
 sx += a[0]; sy += a[1]; sz += a[2]
 p_sum += p; count += 1
 utime.sleep_ms(30)

 x_bias, y_bias = sx / count, sy / count
 z_bias = (sz / count) - 1024.0 # Remove gravity
 baseline_pressure = p_sum / count

 print("\n=====")
 print("STEP 2/2: MAGNETOMETER CALIBRATION (Rotate in Figure-8)")

```

```

print("=====")
min_x, max_x, min_y, max_y = 32767, -32768, 32767, -32768
start = utime.ticks_ms()
while utime.ticks_diff(utime.ticks_ms(), start) < 7000:
 m = read_mag()
 if m:
 if m[0] < min_x: min_x = m[0]
 if m[0] > max_x: max_x = m[0]
 if m[1] < min_y: min_y = m[1]
 if m[1] > max_y: max_y = m[1]
 utime.sleep_ms(20)

mag_x_bias = (max_x + min_x) / 2.0
mag_y_bias = (max_y + min_y) / 2.0

print("=====\\n")

--- NAVIGATION MATH ---
def get_heading():
 m = read_mag()
 if m is None: return 0.0
 mx_c = m[0] - mag_x_bias
 my_c = m[1] - mag_y_bias
 heading = math.degrees(math.atan2(my_c, mx_c))
 return heading + 360.0 if heading < 0 else heading

def get_cardinal(h):
 if h >= 337.5 or h < 22.5: return "N"
 elif 22.5 <= h < 67.5: return "NE"
 elif 67.5 <= h < 112.5: return "E"
 elif 112.5 <= h < 157.5: return "SE"
 elif 157.5 <= h < 202.5: return "S"
 elif 202.5 <= h < 247.5: return "SW"

```

```
elif 247.5 <= h < 292.5: return "W"
```

```
else: return "NW"
```

```
def get_relative_altitude(p_curr):
```

```
 if p_curr is None or baseline_pressure == 0: return 0.0
```

```
 return 44330.0 * (1.0 - math.pow((p_curr / baseline_pressure), 0.1903))
```

```
#
```

```
=====
```

```
=====
```

```
EMBEDDED AI DECISION TREE INFERENCE ENGINE
```

```
(Replace this function with the auto-generated code from Spyder IDE if needed)
```

```
#
```

```
=====
```

```
=====
```

```
def ai_classify_motion(accel, avg, std, p2p):
```

```
 if std <= 1.8450:
```

```
 return 0 # IDLE / SITTING
```

```
 else:
```

```
 if p2p > 82.5000:
```

```
 return 2 # SHAKING / CABLE NOISE
```

```
 else:
```

```
 if accel > (avg * 1.15):
```

```
 return 1 # VALID WALKING
```

```
 else:
```

```
 return 0 # RECOVERY PHASE
```

```
--- MAIN EXECUTION LOOP ---
```

```
def main():
```

```
 global total_steps, total_distance, last_step_time, pos_x, pos_y, pos_z
```

```
 global current_floor, last_heading, prev_activity
```

```
 init_sensors()
```

```
calibrate_system()
```

```
smooth_accel = 0.0
```

```
accel_buffer = []
```

```
print(">>> AI-ENHANCED 3D-PDR SYSTEM ONLINE: START WALKING <<<\n")
```

```
while True:
```

```
 now = utime.ticks_ms()
```

```
 acc = read_accel()
```

```
 if acc:
```

```
 ax, ay, az = acc[0] - x_bias, acc[1] - y_bias, acc[2] - z_bias
```

```
 raw_mag = math.sqrt(ax**2 + ay**2 + az**2)
```

```
 smooth_accel = (ALPHA * raw_mag) + ((1.0 - ALPHA) * smooth_accel)
```

```
 accel_buffer.append(smooth_accel)
```

```
 if len(accel_buffer) > 15: accel_buffer.pop(0)
```

```
 if len(accel_buffer) == 15:
```

```
 # 1. Feature Extraction
```

```
 avg_val = sum(accel_buffer) / 15.0
```

```
 variance = sum((v - avg_val)**2 for v in accel_buffer) / 15.0
```

```
 std_dev = math.sqrt(variance)
```

```
 p2p_delta = max(accel_buffer) - min(accel_buffer)
```

```
 # 2. AI Motion State Classification
```

```
 motion_state = ai_classify_motion(smooth_accel, avg_val, std_dev,
p2p_delta)
```

```
 # Activity Label Update
```

```
 if motion_state == 1: activity = "WALKING"
```

```
elif motion_state == 2: activity = "SHAKING/NOISE"
else: activity = "IDLE/SITTING"
```

```
if activity != prev_activity:
 print(f"\n[ACTIVITY STATE CHANGE]: {activity}")
 prev_activity = activity
```

```
3. Barometric Altitude & Floor Level Update (ICP-20111)
```

```
p_curr = read_baro_pressure()
pos_z = get_relative_altitude(p_curr)
current_floor = int(round(pos_z / FLOOR_HEIGHT_M))
```

```
4. AI-Gated Step Detection Execution
```

```
if motion_state == 1: # Step is ONLY counted if AI verifies WALKING
```

```
 time_diff = utime.ticks_diff(now, last_step_time)
```

```
 if time_diff > STEP_COOLDOWN_MS:
```

```
 total_steps += 1
```

```
 total_distance += STRIDE_LENGTH_M
```

```
 last_step_time = now
```

```
 heading = get_heading()
```

```
 cardinal = get_cardinal(heading)
```

```
Turn Detection Logic
```

```
heading_diff = abs(heading - last_heading)
```

```
if heading_diff > 180: heading_diff = 360 - heading_diff
```

```
if last_heading != 0 and heading_diff > 25:
```

```
 turn = "RIGHT" if (heading - last_heading) % 360 < 180 else "LEFT"
```

```
 print(f" [!] TURN DETECTED: {turn} ({heading_diff:.1f}°)")
```

```
 last_heading = heading
```

```
2D Cartesian Coordinate Update (X, Y)
```

```
rad = math.radians(heading)
```

```
pos_x += STRIDE_LENGTH_M * math.sin(rad)
pos_y += STRIDE_LENGTH_M * math.cos(rad)
```

```
Complete 3D Live Output
```

```
print(f"STEP #{total_steps} | Dir: {cardinal} ({heading:.1f}°) | "
 f"Dist: {total_distance:.1f}m | "
 f"Pos(X,Y): ({pos_x:.2f}m, {pos_y:.2f}m) | "
 f"Height(Z): {pos_z:+.2f}m (Floor {current_floor})")
```

```
utime.sleep_ms(20)
```

```
if __name__ == "__main__":
 main()
```

```
...
```